Manipulation des collections en Swift

420-249-AH — Programmation mobile iOS

1. Pourquoi ces fonctions ?

Swift fournit plusieurs fonctions très utiles pour manipuler les collections :

```
map
filter
sorted
reduce
```

Elles utilisent souvent des closures.

2. map

map transforme chaque élément d'une collection.

Exemple :

```swift
let nombres = [1, 2, 3, 4]
let doubles = nombres.map {
    $0 * 2
}
```

Résultat :

```
Console
[2, 4, 6, 8]
```

3. Exemple avec des chaînes

```swift
let noms = ["alex", "marie", "sam"]
let majuscules = noms.map {
    $0.uppercased()
}
```

Résultat :

```
Console
["ALEX", "MARIE", "SAM"]
```

4. filter

filter conserve seulement les éléments qui respectent une condition.

```
let nombres = [5, 12, 3, 20, 8]
let grands = nombres.filter {
    $0 >= 10
}
```

Résultat :

```
Console
[12, 20]
```

5. Exemple — Véhicules disponibles

```
struct Vehicule {
    let nom: String
    let disponible: Bool
}
```

Puis :

```
let vehiculesDisponibles = vehicules.filter {
    $0.disponible
}
```

6. sorted

sorted trie une collection.

```
let nombres = [8, 2, 10, 4]
let tries = nombres.sorted()
```

Résultat :

```
Console
[2, 4, 8, 10]
```

Ordre décroissant :

```
let tries = nombres.sorted {
    $0 > $1
}
```

7. Trier des objets

```
struct Produit {
    let nom: String
    let prix: Double
}
```

Puis :

```
let produitsTries = produits.sorted {
    $0.prix < $1.prix
}
```

Les produits sont triés du moins cher au plus cher.

8. reduce

reduce combine tous les éléments pour produire une seule valeur.

Exemple :

```
let prix = [10.0, 20.0, 5.0]
let total = prix.reduce(0) {
    $0 + $1
}
```

Résultat :

```
Console
35.0
```

Version courte :

```
let total = prix.reduce(0, +)
```

9. Combiner plusieurs opérations

On peut combiner :

```
let nombres = [1, 2, 3, 4, 5, 6]
let resultat = nombres
    .filter { $0 % 2 == 0 }
    .map { $0 * 10 }
```

Résultat :

```
Console
[20, 40, 60]
```

10. forEach

forEach permet d'exécuter une action pour chaque élément.

```
let noms = ["Alex", "Marie", "Sam"]
noms.forEach {
    print($0)
}
```

Il ressemble à :

```
for nom in noms {
    print(nom)
}
```

11. compactMap

compactMap transforme les éléments et élimine les valeurs nil.

```
let valeurs = ["10", "20", "abc", "40"]
let nombres = valeurs.compactMap {
    Int($0)
}
```

Résultat :

```
Console
[10, 20, 40]
```

Très utile avec les optionnels.

12. first(where:)

Permet de trouver le premier élément correspondant à une condition.

```
let nombres = [2, 5, 12, 18]
let resultat = nombres.first {
    $0 > 10
}
```

Résultat :

```
Console
Optional(12)
```

Le résultat est un optional.

13. contains

Vérifie si une collection contient une valeur.

```
let roles = ["admin", "user", "guest"]
let existe = roles.contains("admin")
```

Résultat :

```
Console
true
```

Avec une condition :

```swift
let existe = nombres.contains {
    $0 > 100
}
```

14. Résumé

Fonction	Rôle
`map`	Transformer
`filter`	Sélectionner
`sorted`	Trier
`reduce`	Combiner
`forEach`	Parcourir et exécuter une action
`compactMap`	Transformer et supprimer les nil
`first`	Trouver le premier élément
`contains`	Vérifier la présence

15. Exemple complet — Catalogue de produits

```swift
struct Produit {
    let nom: String
    let prix: Double
    let disponible: Bool
}

let produits = [
    Produit(nom: "iPhone", prix: 1200, disponible: true),
    Produit(nom: "MacBook", prix: 2200, disponible: false),
    Produit(nom: "AirPods", prix: 250, disponible: true)
]
```

Produits disponibles :

```swift
let disponibles = produits.filter {
    $0.disponible
}
```

Tri par prix :

```swift
let tries = disponibles.sorted {
    $0.prix < $1.prix
}
```

Récupérer les noms :

```
let noms = tries.map {
    $0.nom
}
```

Total :

```
let total = disponibles
    .map { $0.prix }
    .reduce(0, +)
```

16. Exercice — Gestion de trajets

Créer :

```
struct Trajet {
    let nom: String
    let distance: Double
    let termine: Bool
}
```

Créer ensuite plusieurs trajets.

À l'aide de filter, sorted, map et reduce :

- Récupérer seulement les trajets terminés
- Les trier par distance
- Récupérer uniquement leurs noms
- Calculer la distance totale des trajets terminés

17. Exercice — Premier trajet de plus de 100 km

Trouver le premier trajet dont la distance dépasse :

```
100
```

km, avec :

```
first(where:)
```